GUEST SESSION

# Modern Python

## uv, Clean Code, and Coding with AI

A hands-on refresher

Somto Muotoe

October 2026

`github.com/smuotoe/mcda-python-refresher`

# Install uv now

Do this while everyone settles in. It takes about a minute.

### Windows (recommended)

```
winget install --id=astral-sh.uv -e
```

Asked to accept the source agreements? Type `Y`. "winget" not recognised? Install **App Installer** from the Microsoft Store.

### macOS / Linux

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Or, with Homebrew: `brew install uv`

### Then open a NEW terminal

```
uv --version
git --version
```

No git? You can still get the exercises later: on GitHub, **Code** > **Download ZIP**.

**Windows fallback** (PowerShell installer):
`powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"`

**Last resort** (Python already installed): `python -m pip install uv`, then type `python -m uv` wherever the slides say `uv`.

Sources: docs.astral.sh/uv/getting-started/installation

ABOUT ME

# Hi, I'm Somto

Somto Muotoe

# Agenda

`0:00 - 0:10`
### Install + intro
uv on every laptop, who is in the room

`0:10 - 0:15`
### How things have shifted
Python packaging and AI coding, then and now

`0:15 - 0:40`
### uv
Projects, dependencies, Python versions, scripts, tools

`0:40 - 1:15`
### Functions, docstrings, classes
Hands-on: three exercises

`1:15 - 1:27`
### Coding with AI
Specify, generate, verify, explain

`1:27 - 1:30`
### Wrap-up
Checklist, resources, questions

Laptops open the whole time. Setup steps, exercises and a uv cheat sheet are in the repo README.

# Quick poll: show of hands

### How do you manage Python environments today?

- pip + venv
- conda
- poetry
- uv
- whatever works

### Do you use AI to write code?

- chat (ChatGPT, Claude, ...)
- autocomplete (Copilot, ...)
- agents (Claude Code, Codex, ...)
- not yet

No wrong answers. "Whatever works" is the most honest one.

# Python packaging: one problem at a time

Every few years a new tool fixed one piece. uv now covers all of them in one tool.

Sep 2007
**virtualenv**
isolated environments

Oct 2008
**pip**
the package installer

Sep 2012
**venv in the standard library**
Python 3.3, PEP 405

Sep 2012
**conda**
packages beyond Python

May 2016
**pyproject.toml**
PEP 518: one config file

Jan 2017
**Pipenv**
Pipfile + lock file

Feb 2018
**Poetry**
dependencies, lock, build

Nov 2020
**PEP 621**
project metadata in pyproject.toml

Jan 2024
**PEP 723**
inline script metadata

Feb 2024
**uv**
a fast pip replacement

Aug 2024
**uv, project manager**
projects, lock files, Pythons, tools

Mar 2025
**PEP 751**
a standard lock file (pylock.toml)

Sources: PyPI release histories for virtualenv, pip, Pipenv, Poetry · Python 3.3.0 · conda changelog · PEP 405 · PEP 518 · PEP 621 acceptance · PEP 723 · PEP 751 · astral.sh/blog/uv · astral.sh/blog/uv-unified-python-packaging

# AI and coding: autocomplete, then chat, then agents

### AUTOCOMPLETE

AI finishes the line you are typing

`Jun 2021` GitHub Copilot technical preview

### CHAT

You ask, you copy, you paste

`Nov 2022` ChatGPT

`Mar 2023` GPT-4

### AGENTS

AI edits files and runs commands in your terminal

`Feb 2025` "vibe coding" (Karpathy)

`Feb 2025` Claude Code preview (GA May 2025)

`Apr 2025` OpenAI Codex CLI

`Jun 2025` Gemini CLI

`Aug 2025` AGENTS.md rules files

Each step hands the AI more of the work, and leaves you more to check.

Sources: GitHub blog (Copilot) · OpenAI (ChatGPT) · OpenAI (GPT-4) · Karpathy on X (see also) · Anthropic (Feb 2025) · Anthropic (May 2025) · OpenAI (Codex CLI), github.com/openai/codex · Google (Gemini CLI) · Linux Foundation (AGENTS.md)

# What this changes for you

MY COURSE PLAN, 2023

"...the bulk of the code can be written by GPT-4 or other coding assistants"

2026

Agents write whole features, across many files.

Typing code is cheap now. The scarce skills are:

**Specify**
say exactly what you want

**Verify**
check what you got

**Explain**
defend every line

Type hints and docstrings are how you talk to **both** the humans and the AI that read your code.

84% of developers use or plan to use AI tools, but more distrust their accuracy (46%) than trust it (33%). Stack Overflow survey, 2025

19% slower with AI in an early-2025 trial, while the developers believed AI made them 20% faster. METR now says these results no longer reflect current tools. METR, 2025-26

Sources: survey.stackoverflow.co/2025/ai · METR (July 2025 study) · METR (Feb 2026 update)

PART 3 · 0:15 - 0:40

# uv: one tool for Python

All the ways I use it, from installing Python to one-off scripts

# What uv replaces

| You used to need | With uv |
|---|---|
| pip | `uv add`, `uv pip install` |
| pip-tools | `uv lock`, `uv sync` |
| venv / virtualenv | automatic (or `uv venv`) |
| pipx | `uvx`, `uv tool install` |
| pyenv | `uv python install` |
| poetry | `uv init`, `uv add`, `uv lock` |

### One fast binary

Written in Rust by Astral, the makers of the Ruff linter.

### 10-100x

faster than pip, according to Astral's docs. We'll test that claim in a minute.

Sources: docs.astral.sh/uv · astral.sh

# The race: pip vs uv

Windows PowerShell shown. It has no `&&`, so run one line at a time. Try it if the Wi-Fi allows.

## Round 1: venv + pip

```
python -m venv .venv
.venv\Scripts\activate
Measure-Command { pip install pandas numpy matplotlib scikit-learn requests }
```

## Round 2: uv, cold (no cache)

```
deactivate
uv init race-uv
cd race-uv
Measure-Command { uv add --no-cache pandas numpy matplotlib scikit-learn requests }
```

## Round 3: uv, warm cache (a fresh project, without `--no-cache`)

```
cd ..
uv init race-uv2
cd race-uv2
Measure-Command { uv add pandas numpy matplotlib scikit-learn requests }
```

macOS/Linux: `source .venv/bin/activate`, and `time pip install ...` instead of `Measure-Command { ... }`. New to uv? Your cache starts empty, so repeat round 3 in one more fresh project to see the warm time.

# Projects: uv init and uv run

```
uv init --no-package my-app
cd my-app
uv run main.py
```

```
my-app/
├── .git/              a git repo, ready to commit
├── .gitignore
├── .python-version    which Python this project uses
├── README.md
├── main.py
├── pyproject.toml     name, version, dependencies
├── .venv/             created by the first uv run
└── uv.lock            created by the first uv run
```

```toml
# pyproject.toml
[project]
name = "my-app"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.13"
dependencies = []
```

> No activation, ever. `uv run` creates `.venv` and `uv.lock` when needed, syncs them, then runs your code.

Plain `uv init my-app` (current uv) makes a packaged app instead: code in `src/my_app/`, run with `uv run my-app`. `--no-package` keeps the simple `main.py` layout.

Sources: docs.astral.sh/uv/concepts/projects/init

# Dependencies

| Command | What it does |
| --- | --- |
| `uv add httpx` | add to pyproject.toml ( `httpx>=0.28.1` ), update uv.lock, install |
| `uv add "polars>=1.0"` | add with a version constraint (quote it: `>` is a shell redirect) |
| `uv add --dev ruff` | development-only tool, kept out of your app's dependencies |
| `uv remove httpx` | remove it from pyproject.toml, uv.lock and .venv |
| `uv sync` | make .venv match the lock file exactly |
| `uv lock --upgrade-package httpx` | upgrade one package in uv.lock, leave the rest alone |
| `uv tree` | show what depends on what |

**Commit** `pyproject.toml` **and** `uv.lock` . A teammate clones, runs `uv sync` , and gets the identical environment: same Python version, same package versions.

Sources: docs.astral.sh/uv/concepts/projects/dependencies · docs.astral.sh/uv/concepts/projects/sync

# Python versions, no pyenv or conda

| Command | What it does |
| --- | --- |
| `uv python install 3.13` | download and install a Python version |
| `uv python pin 3.13` | write `.python-version`, so this project always uses 3.13 |
| `uv python list` | show installed versions and the ones available to download |
| `uv run --python 3.12 python --version` | run with another version, just this once |

**You rarely need** `install`. When a command needs a Python you don't have, uv downloads it automatically.

**Inside a project,** `--python` must satisfy `requires-python` in pyproject.toml. 3.12 in a `>=3.13` project is an error.

Sources: docs.astral.sh/uv/concepts/python-versions · docs.astral.sh/uv/guides/install-python

# Scripts that bring their own dependencies

PEP 723: dependencies live inside the file

```
# /// script
# requires-python = ">=3.13"
# dependencies = [
#     "httpx>=0.28.1",
#     "rich>=15.0.0",
# ]
# ///
import httpx
from rich import print

url = "https://pypi.org/pypi/uv/json"
version = httpx.get(url).json()["info"]["version"]
print(f"[bold green]Latest uv:[/] {version}")
```

```
uv init --script check.py
uv add --script check.py httpx rich
uv run check.py
```

Runs from any folder: no project, no venv to manage. uv builds a cached environment for the script.

**macOS/Linux:** add this as the first line, run `chmod +x check.py`, then just `./check.py`

```
#!/usr/bin/env -S uv run --script
```

**In the repo:** `demos/pypi_check.py` checks that package names exist on PyPI. Use it before you install a package an AI suggested:

```
uv run demos/pypi_check.py requests polars definitely-not-a-real-pkg
```

Sources: PEP 723 · docs.astral.sh/uv/guides/scripts

# Borrow a package or a tool for one command

A package for one run: `uv run --with`

```
uv run --with rich python -c "import rich; rich.print('[bold green]hello')"
uv run --with jupyter jupyter lab   # Jupyter on this project, not added as a dependency
uv add --dev ipykernel              # notebooks in VS Code: pick .venv as the kernel
```

A tool: `uvx` and `uv tool`

```
uvx pycowsay "hello"     # run a tool once, in a throwaway environment
uvx ruff check .         # lint this folder without installing Ruff
uv tool install ruff     # install for good: ruff is now on your PATH
uv tool list             # what's installed
uv tool upgrade --all    # upgrade every tool
```

`uvx` is short for `uv tool run`, like `npx` for Node. Each tool gets its own environment, so tools never break your projects.

Sources: docs.astral.sh/uv/guides/tools · docs.astral.sh/uv/guides/integration/jupyter

# Old projects and housekeeping

| Command | When |
| --- | --- |
| `uv venv` then `uv pip install -r requirements.txt` | an old pip project: same workflow, much faster |
| `uv init` then `uv add -r requirements.txt` | migrate it to a uv project |
| `uv export --format requirements-txt -o requirements.txt` | a platform still wants requirements.txt |
| `uv self update` | update uv (standalone installer only) |
| `winget upgrade --id=astral-sh.uv -e` | update uv if you installed it with winget |
| `uv cache prune` | free disk space: remove unused cache entries |
| `uv cache clean` | wipe the whole cache (the next installs download again) |
| `uv format` | format your project's code with Ruff (a preview feature, uv 0.8.13+) |

Sources: docs.astral.sh/uv/pip · docs.astral.sh/uv/getting-started/installation · docs.astral.sh/uv/concepts/cache · docs.astral.sh/uv/concepts/preview

# uv cheat sheet

| I want to... | Command |
|---|---|
| start a project | `uv init --no-package my-app` |
| run code | `uv run main.py` |
| add a dependency | `uv add httpx` |
| add a dev-only tool | `uv add --dev ruff` |
| remove a dependency | `uv remove httpx` |
| match the lock file | `uv sync` |
| upgrade one package | `uv lock --upgrade-package httpx` |
| see the dependency tree | `uv tree` |
| update uv | `uv self update` / `winget upgrade` |

| I want to... | Command |
|---|---|
| install a Python | `uv python install 3.13` |
| pin a project's Python | `uv python pin 3.13` |
| try another Python once | `uv run --python 3.12 ...` |
| write a one-file script | `uv init --script check.py` |
| give a script dependencies | `uv add --script check.py rich` |
| use a package once | `uv run --with rich ...` |
| run a tool once | `uvx ruff check .` |
| install a tool | `uv tool install ruff` |
| use a requirements.txt | `uv pip install -r requirements.txt` |

Photograph this one. The repo README has a cheat sheet too.

# Your turn: hello, uv (5 min)

```
uv init --no-package hello-uv
cd hello-uv
uv add rich
# replace main.py with the code on the right
uv run main.py
uvx pycowsay "uv rocks"
```

**Bonus:** `uv run --python 3.12 main.py`

Got an error? Read it. Then change this line in pyproject.toml and run it again:

```
requires-python = ">=3.12"
```

```python
from rich.console import Console
from rich.table import Table


def main() -> None:
    table = Table(title="My Python toolbox")
    table.add_column("Tool", style="cyan")
    table.add_column("What it does")
    table.add_row("uv", "projects and dependencies")
    table.add_row("ruff", "linting and formatting")
    table.add_row("git", "version control")
    Console().print(table)


if __name__ == "__main__":
    main()
```

# Get the exercises

```
git clone https://github.com/smuotoe/mcda-python-refresher
cd mcda-python-refresher
uv sync
uv run exercises/functions.py
```

> One command, my exact environment. `uv sync` reads `uv.lock` and `.python-version`, and downloads Python 3.13 if you don't have it.

A `NotImplementedError: TODO` on the first run is expected: that's your first task.

No git? On the GitHub page: **Code** > **Download ZIP**, unzip, and `cd` into the folder.

```
exercises/
  functions.py     Exercise 1
  docstrings.py    Exercise 2
  classes.py       Exercise 3
  spec_first.py    Exercise 4 (with AI)
demos/
  pypi_check.py
AGENTS.md
pyproject.toml
uv.lock
```

Each exercise: fill in the TODOs, run the file, compare with the `# expected:` comments.

# Functions: anatomy of a good one

```python
def average(scores: list[float], drop_lowest: bool = False) -> float:
    """Return the mean score, optionally ignoring the lowest one."""
    if not scores:
        raise ValueError("scores must not be empty")
    if drop_lowest and len(scores) > 1:
        scores = sorted(scores)[1:]
    return sum(scores) / len(scores)
```

Name says what it returns; typed parameters and a return type say what goes in and out.

Docstring: one line on what it does.

One function, one job. Raise early on bad input.

Return values, don't print them, so other code can use the result.

**Modern type hints**

```python
names: list[str]
counts: dict[str, int]
nickname: str | None = None
def first[T](items: list[T]) -> T: ... # Python 3.12+
```

Sources: docs.python.org/3/library/typing · PEP 695 (type parameter syntax, Python 3.12)

# Pitfall: mutable default arguments

```python
def add_tag(tag: str, tags: list[str] = []) -> list[str]:
    tags.append(tag)
    return tags

add_tag("a")  # ['a']
add_tag("b")  # ['a', 'b']  the default list is shared!
```

**Why:** the default `[]` is created once, when `def` runs. Every call that uses the default gets the *same* list.

**Fix:** default to `None`, and create a new list inside the function.

Linters catch this: Ruff reports it as rule `B006`.

Sources: Python FAQ: why are default values shared between objects? · Ruff rule B006

# Exercise 1: functions (8 min)

File: `exercises/functions.py`

### `slugify(title: str) -> str`

- lowercase
- every run of characters that aren't letters or digits becomes one `-`
- strip `-` from both ends
- no accent handling

```
slugify("Hello, World!")
-> "hello-world"
```

### `add_tag(tag, tags=[])`

- run the file first
- the second call returns `['python', 'uv']` instead of `['uv']`
- fix the mutable default

### `chunk[T](items, size)`

- split into lists of length `size`
- the last chunk may be shorter
- `ValueError` if `size` < 1

```
chunk([1, 2, 3, 4, 5], 2)
-> [[1, 2], [3, 4], [5]]
```

**Check:** `uv run exercises/functions.py` and compare with the `# expected:` comments

# Docstrings: written for humans and for AI

## Why they matter more now

Shown by `help()` and by IDE hovers

Turned into documentation by doc generators

Read by AI assistants as the spec for your code

Frameworks such as LangChain and MCP build an LLM's tool descriptions straight from your **type hints and docstrings**

## Google style

**Summary line**, then **Args**, **Returns**, **Raises**

```python
def apply_discount(price: float, percent: float) -> float:
    """Apply a percentage discount to a price.

    Args:
        price: Original price.
        percent: Discount percentage between 0 and 100.

    Returns:
        The discounted price, rounded to 2 decimal places.

    Raises:
        ValueError: If percent is outside 0-100.
    """
```

Sources: Google Python Style Guide 3.8 · LangChain: tools · MCP: build a server

# Exercise 2: docstrings (5 min)

File: `exercises/docstrings.py`

## The task

Both functions work, but have no docstrings. Add a Google-style docstring to each:

- `apply_discount(price, percent)`
- `split_full_name(full_name)`

Summary line, then Args, Returns and Raises. Read the code to find what each one raises.

## Check

```
uv run exercises/docstrings.py
```

It prints each docstring. `None` means you're not done yet.

**Tip:** hover over the function name in VS Code to see your docstring as others will.

# Classes: state plus behaviour

Use a class when data and the operations on it belong together. Otherwise, plain functions.

Anatomy: a simple `__init__`, methods, `@property` for computed values, dunder methods such as `__len__` and `__repr__`.

`@dataclass` for data containers:

```python
@dataclass(frozen=True)
class Song:
    title: str
    seconds: int
```

`frozen=True`: immutable and hashable.

Composition over inheritance: a Playlist *has* Songs; it doesn't extend `list`.

```python
class Playlist:
    def __init__(self, name: str) -> None:
        self.name = name
        self._songs: list[Song] = []

    def add(self, song: Song) -> None:
        self._songs.append(song)

    @property
    def total_seconds(self) -> int:
        return sum(song.seconds for song in self._songs)

    def __len__(self) -> int:
        return len(self._songs)

    def __repr__(self) -> str:
        return f"Playlist({self.name!r}, {len(self)} songs)"
```

Docstrings omitted to fit the slide.

Sources: docs.python.org/3/library/dataclasses

# Exercise 3: classes (8 min)

File: `exercises/classes.py`

## Given: `Product`

```python
@dataclass(frozen=True)
class Product:
    name: str
    price_cents: int
```

`@dataclass` gives you `__init__`, `__repr__` and `__eq__` for free. `frozen=True` makes it hashable, so a Product can be a dictionary key.

## Your job: `ShoppingCart`

- `add(product, quantity=1)`: `ValueError` if quantity < 1
- `remove(product)`: removes it entirely; `KeyError` if absent
- `total_cents`: a property
- `__len__`: total number of units

`uv run exercises/classes.py`

Expected: `3`, `1175`, then `1 275`

PART 5 · 1:15 - 1:27

# Specify, Generate, Verify, Explain

Coding with AI without losing ownership of your code

# The loop

1
## Specify

Signature, type hints and a docstring that say exactly what you want.

2
## Generate

Any AI tool: chat, Copilot, Claude Code, Codex, Gemini CLI.

3
## Verify

Run it, compare with the expected output, and read the code.

4
## Explain

You can defend every line to a colleague.

**If you can't explain it, you don't own it.**

Not happy at step 3 or 4? Go back to step 1 and tighten the spec.

# Exercise 4: spec first (7 min)

File: `exercises/spec_first.py` · `parse_duration(text: str) -> int` returns minutes

**Step 1: vague prompt.** Ask any AI: *"Write a Python function that parses durations like 1h30m."* Paste its answer over the stub, run the file, and compare.

**Step 2: full spec.** Give the AI the whole docstring from the file (the rules). Paste, rerun, compare again.

**Step 3: explain** every line to your neighbour.

**Lesson:** wherever your spec is ambiguous, the AI guesses.

`uv run exercises/spec_first.py`

```
  '1h30m' -> 90
     '2h' -> 120
    '45m' -> 45
'1H 30m' -> 90
 '1h90m' -> ValueError
  '1.5h' -> ValueError
 '30m1h' -> ValueError
      '' -> ValueError
```

Expected output. The last four are where a vague prompt has to guess.

# Guardrails

## AGENTS.md: rules your agent reads

A plain Markdown file in the repo root that coding agents read before they work. From ours:

```
- Use uv: `uv add <package>` to add
  dependencies and `uv run <file>` to run
  code. Never call pip directly.
- Don't add a dependency without asking,
  and check that it exists on PyPI first.
```

## Check that a package exists before `uv add`

AIs sometimes invent package names. Attackers can register those names and fill them with malware ("slopsquatting").

A study of 16 code LLMs and 576,000 code samples found hallucinated packages in at least 5.2% (commercial) and 21.7% (open-source) of cases: 205,474 unique fake names.

```
uv run demos/pypi_check.py some-package
```

## Secrets stay out of git

API keys go in `.env`, and `.env` goes in `.gitignore`. Read them from environment variables.

## Mind what you paste

Don't paste private data, client code or credentials into public chat tools.

Sources: agents.md · Spracklen et al., "We Have a Package for You!", USENIX Security 2025 (arXiv 2406.10279)

# Checklist for every project

- [ ] A **uv project**, with `pyproject.toml` and `uv.lock` committed
- [ ] A **README** with setup and run steps (`uv sync`, `uv run ...`)
- [ ] **Small functions** with type hints and Google-style docstrings
- [ ] **No secrets in code**: `.env` for keys, and `.env` in `.gitignore`

This is what I'll look for in January.

# Resources

## uv

docs.astral.sh/uv
guides for projects, scripts, tools and Python versions

PEP 723: inline script metadata
the `# /// script` block

## Writing clear Python

Python typing documentation
every type hint, with examples

Google Python Style Guide: docstrings
Args, Returns, Raises, with examples

> **Today's exercises and slides**
>
> `github.com/smuotoe/mcda-python-refresher`

# Thank you

Questions?

See you in January.

`github.com/smuotoe/mcda-python-refresher`